

SKILLOPT: Executive Strategy for Self-Evolving Agent Skills

Yifan Yang^{1,*,\dagger} Ziyang Gong^{2,*} Weiquan Huang^{3,*} Qihao Yang^{2,*} Ziwei Zhou^{4,*}
Zisu Huang^{4,*} Yan Li² Xuemei Gao¹ Qi Dai¹ Bei Liu¹
Kai Qiu¹ Yuqing Yang¹ Dongdong Chen¹ Xue Yang^{2,\dagger} Chong Luo¹

¹ Microsoft ² Shanghai Jiao Tong University ³ Tongji University ⁴ Fudan University

Agent skills today are hand-crafted, generated one-shot, or evolved through loosely controlled self-revision—none of which behaves like a deep-learning optimizer for the skill, and none of which reliably improves over its starting point under feedback. We argue the skill should instead be *trained* as the external state of a frozen agent, with the same discipline that makes weight-space optimization reproducible. SKILLOPT is, to our knowledge, the first systematic *controllable* text-space optimizer for agent skills: a separate optimizer model turns scored rollouts into bounded add/delete/replace edits on a single skill document, and an edit is accepted only when it strictly improves a held-out validation score. A textual learning-rate budget, rejected-edit buffer, and epoch-wise slow/meta update make skill training stable while adding zero inference-time model calls at deployment. Across six benchmarks, seven target models, and three execution harnesses (direct chat, Codex, Claude Code), SKILLOPT is best or tied on **all 52 evaluated (model, benchmark, harness) cells** and beats every per-cell competitor among human, one-shot LLM, Trace2Skill, TextGrad, GEPA, and EvoSkill skills. On GPT-5.5 it lifts the average no-skill accuracy by +23.5 points in direct chat, by +24.8 inside the Codex agentic loop, and by +19.1 inside Claude Code. Transfer experiments further show that optimized skill artifacts retain value when moved across model scales, between Codex and Claude Code execution environments, and to a nearby math benchmark without further optimization.

Code: <https://aka.ms/SkillOpt>

Correspondence: yifanyang@microsoft.com, yangxue2019-sjtu@sjtu.edu.cn

* Equal contribution. ^{\dagger} Corresponding authors.

1 Introduction

Frontier language models are increasingly deployed as agents, from single-prompt callers to multi-step execution harnesses with tools, files, and verifiers [1–4]. In such settings, domain adaptation is no longer only about model weights or prompts: it also requires improving the *procedures* by which the agent gathers evidence, calls tools, follows domain conventions, and formats outputs [5, 6]. Agent skills provide a natural interface for this procedural adaptation [7, 8]: a skill is a portable natural-language artifact that packages procedures, domain heuristics, tool policies, output constraints, and failure modes, letting a frozen agent adapt through external text.

If the recurring object of adaptation is the agent’s procedure, the skill document itself should be trainable. Yet weight adaptation is often unavailable for closed frontier models and expensive

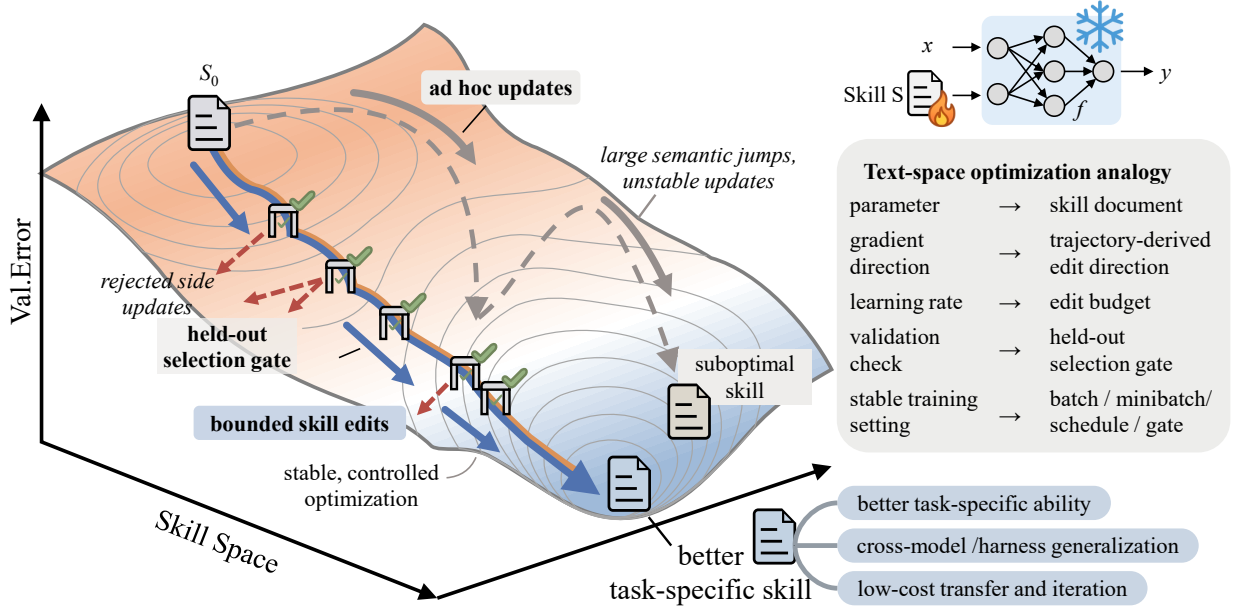


Figure 1 Overview of SKILLOPT. The target model executes tasks with a current skill, an additional frontier optimizer model converts trajectories into bounded add/delete/replace skill edits, and a held-out gate accepts only edits that improve validation performance. Accepted edits are exported as a reusable skill artifact, while rejected edits become negative feedback for later updates.

for open ones, while manually written or one-shot skills are brittle under a target domain or harness. Recent systems convert execution experience into reusable textual artifacts—distilling trajectory lessons, refining skill folders via failure analysis, building domain-specific skill libraries, or optimizing prompts from trajectory feedback [9–13]—but leave open a more basic question: if skills are the adaptation layer, how should they be optimized? Our key idea is to treat skill editing as a controllable domain-adaptation process, with the skill document as the external state, an additional frontier model as the optimizer, and training-style controls over evidence, step size, validation, and update direction.

We introduce SKILLOPT, a text-space optimizer for agent skills. Given a target domain, an initial skill, and the model being adapted, SKILLOPT repeatedly samples trajectory batches, analyzes successes and failures, and asks a frontier optimizer model to propose structured add/delete/replace edits. It then aggregates and ranks candidate edits under a textual learning-rate budget, applies a bounded update to the skill document, and evaluates the candidate skill on a held-out selection split before accepting it. Rejected edits are retained as negative feedback, while the epoch-wise slow/meta update preserves longer-horizon regularities. Figure 1 gives a schematic view of this loop. The deployed output is a compact `best_skill.md` file of roughly 300–2,000 tokens, with the adapted model and execution harness remaining fixed.

The deep-learning analogy is operational rather than decorative. Rollout and reflection batch sizes control the noise in the evidence used for each edit; the textual learning rate and schedule control how far one skill version is allowed to move from the previous one; the held-out gate plays the role of validation; and the epoch-wise slow/meta update acts like a momentum term, carrying stable editing directions across epochs. This stability is crucial: if consecutive skill revisions move too far or in inconsistent directions, rejected edits and previous accepted edits no longer provide a meaningful optimization history. With bounded, validation-gated updates, each revision remains close enough to the last one that later optimizer calls can learn from what helped, what failed, and what should be preserved.

We conduct, to our knowledge, the first systematic study of skill optimization as a domain-adaptation training method for frontier agents. We evaluate SKILLOPT on six benchmarks covering QA, spreadsheets, documents, math, and embodied decision making, across seven target models

from frontier-scale GPT to small-scale Qwen, and under three execution modes (direct chat, Codex harness, Claude Code harness). Out of 52 evaluated (model, benchmark, harness) cells, SKILLOPT is the best or tied-best measured method on all 52. With GPT-5.5 in direct chat, it lifts SearchQA from 77.7 to 87.3, SpreadsheetBench from 41.8 to 80.7, OfficeQA from 33.1 to 72.1, DocVQA from 78.8 to 91.2, LiveMathematicianBench from 37.6 to 66.9, and ALFWorld from 83.6 to 95.5 (a +23.5 point average gain over no skill), and it also beats the strongest *per-cell* baseline drawn from human-written, one-shot LLM, Trace2Skill, TextGrad, GEPA, and EvoSkill skills by +5.4 points on average. The same optimization interface is effective inside Codex-style and Claude Code-style execution loops, lifting GPT-5.5 by +24.8 and +19.1 points over no skill respectively, and outperforming EvoSkill by +14.0 and +3.2 points.

The learned artifacts also transfer beyond the exact training setting. A SpreadsheetBench skill trained on GPT-5.4 improves every smaller GPT variant we test; a Codex-trained spreadsheet skill transfers to Claude Code with a +59.7 point gain; and an OlympiadBench skill yields positive gains on Omni-MATH [14]. These transfer results are important for the paper’s application value: a skill can be optimized once, audited as text, and reused across related models, harnesses, or tasks without changing model weights. Our ablations explain why this works. Bounded textual learning outperforms uncontrolled rewriting, held-out gating prevents harmful proposals from accumulating, the rejected-step buffer converts failed edits into negative feedback, and the epoch-wise slow/meta update improves long-horizon refinement without bloating the deployed skill. Finally, per-benchmark case studies show that the learned skills remain compact (300–2,000 tokens after only 1–4 accepted edits), inspectable, and procedural rather than instance-specific.

Our contributions are as follows:

- We formulate agent-skill learning as optimization over an external natural-language state and introduce SKILLOPT, a harness-agnostic optimizer with rollout batches, reflection minibatches, add/delete/replace edits, textual learning rates, schedules, held-out acceptance, rejected-edit buffers, and epoch-wise slow/meta update.
- We provide a broad empirical study across six benchmarks, seven target models, and three execution harnesses, showing that SKILLOPT is best or tied-best on 52 of 52 cells and outperforms no-skill, human-skill, one-shot LLM-skill, prompt-optimization (TextGrad, GEPA), and skill-evolution (Trace2Skill, EvoSkill) baselines under every model.
- We validate the optimization design through component ablations and three forms of transfer (cross-model, cross-harness, cross-benchmark), showing that the exported skill artifact is compact, reusable, and deployable without model-weight updates.

2 Related Work

Prompt auto tuning and agent-configuration search. GEPA demonstrates that trajectory feedback can guide reflective prompt evolution and outperform reinforcement learning on several language-agent tasks [13]. ABSTRAL and EvoTest extend this idea from single prompts to multi-agent design documents and test-time agentic system evolution without gradients or fine-tuning [15, 16]. By treating language artifacts as optimizable objects, these methods can directly exploit execution feedback, but they mainly target prompts, system designs, or full configurations rather than reusable domain adaptation. SKILLOPT instead optimizes a persistent skill document that can be trained, validated, exported, and reused with the adapted model, applying language-level controllability to a stable procedural skill state.

Skill construction and skill evolution. SkillsBench and the SoK on agentic skills frame skills as reusable procedural knowledge, covering tool policies, applicability conditions, execution

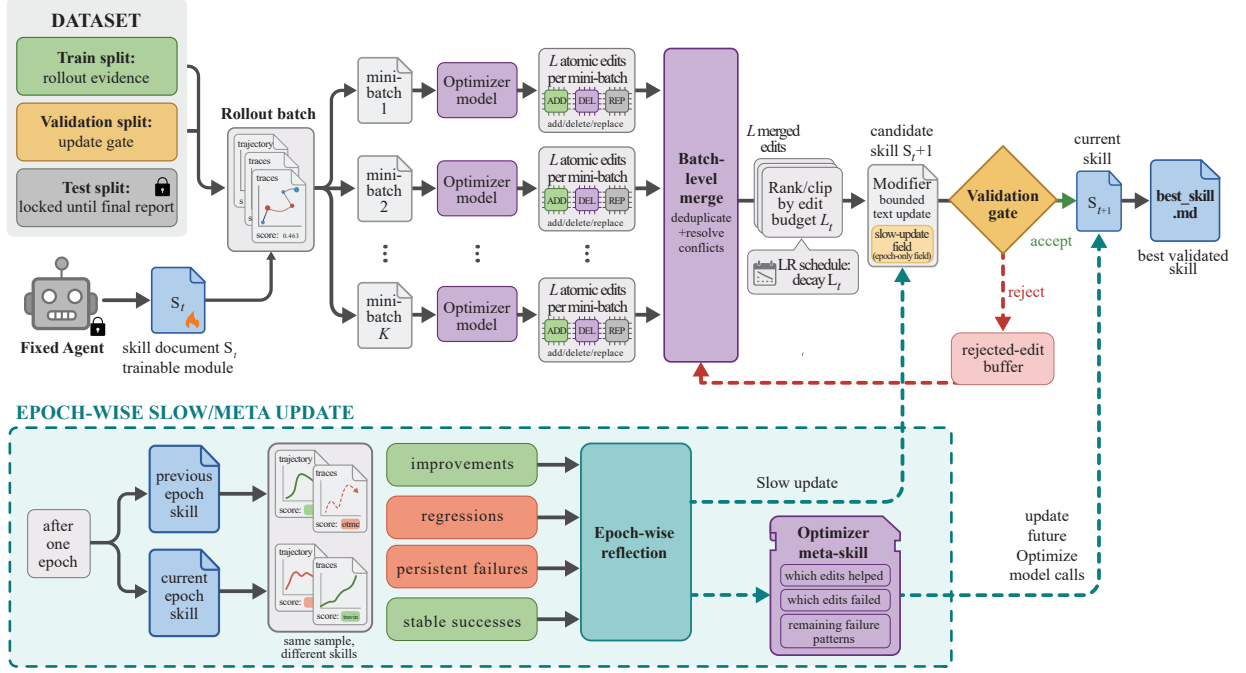


Figure 2 Pipeline of SKILLOPT. A frozen target model executes a rollout batch with the current skill; an optimizer model performs minibatch reflection over successes and failures, proposes bounded add/delete/replace edits, merges and ranks them under a scheduled edit budget, and accepts the candidate skill only through a held-out validation gate. Across epochs, the slow/meta update retains longer-horizon lessons without changing the target model.

routines, and supporting resources [7, 8]. Prior systems construct such skills from lifelong experience, trajectory lessons, skill knowledge bases, or heterogeneous domain resources [9, 12, 17–19], and further refine them through failure analysis, creation-evaluation-revision loops, co-evolving generators and verifiers, collective updates, or reinforcement learning [10, 11, 20–26]. While these works emphasize skill discovery, repository growth, sharing, evolutionary search, or policy optimization, SKILLOPT studies a narrower problem: how to train one compact domain skill with deep-learning-style controls such as trajectory batches, reflection minibatches, textual learning rates, validation gates, rejected-edit buffers, and slow/meta updates. This yields a controlled and auditable procedure for producing a portable `best_skill.md` without changing model weights.

3 Method

3.1 Problem Setup

A skill s is a natural-language policy inserted into the agent context before execution, consistent with recent work treating skills as reusable procedural knowledge for agents [7, 8]. In direct-chat benchmarks, it is prepended to the system or developer instruction; in tool-use harnesses, it becomes persistent procedural memory. We use M to denote the frozen target model whose behavior is being adapted through skill optimization. For a harness h , task x , and skill s , execution produces a trajectory τ and a scalar score r :

$$(\tau(s), r(s)) = h(M, x, s), \quad r(s) \in [0, 1]. \quad (1)$$

Given train, selection, and test splits $D_{\text{tr}}, D_{\text{sel}}, D_{\text{test}}$, SKILLOPT uses D_{tr} to generate a set of

candidate skills $\mathcal{C}(D_{\text{tr}})$, selects the best skill on D_{sel} , and reports the final performance on D_{test} :

$$s_{\text{sel}}^* = \arg \max_{s \in \mathcal{C}(D_{\text{tr}})} \frac{1}{|D_{\text{sel}}|} \sum_{x \in D_{\text{sel}}} r(s), \quad (2)$$

$$\text{Test}(s_{\text{sel}}^*) = \frac{1}{|D_{\text{test}}|} \sum_{x \in D_{\text{test}}} r(s_{\text{sel}}^*). \quad (3)$$

The training split supplies experience, the selection split gates updates, and the test split is used only for final reporting. The optimizer state contains the current skill, the best validation-gated skill, cached skill hashes, an epoch-local rejected-step buffer, and optional slow/meta-update state. Only the best accepted skill is exported as `best_skill.md`.

3.2 Forward Pass: Rollout Evidence

At each optimization step, the target model runs a rollout batch from D_{tr} with the current skill. The harness records task metadata, messages, tool calls, observations, command outputs, final answers, verifier feedback, and benchmark-specific context such as spreadsheet previews, document references, or compact execution traces. This batch is the evidence unit: small batches update quickly but noisily, while larger batches expose more recurring patterns before the skill changes. The implementation also supports accumulation, where several rollout batches are reflected on separately and merged into one update, decoupling execution throughput from update frequency.

3.3 Backward Pass: Minibatch Reflection

The optimizer model turns trajectories into skill edits, following the broader line of trajectory-driven reflection and prompt evolution [13, 27, 28]. It first separates failures from successes and partitions each group into reflection minibatches. This matters because single trajectories often produce anecdotal fixes, while minibatches expose reusable procedural errors: the agent consistently searches the wrong source, writes an answer in the wrong format, or fails to verify a tool result. Failure minibatches propose missing or corrective rules; success minibatches preserve behaviors that already work. Each reflection returns structured add/delete/replace edits, or in rewrite mode a small set of rewrite suggestions.

Local proposals are merged hierarchically by first consolidating failure- and success-driven edits separately, then combining them with priority on failure corrections. This step filters duplicate, contradictory, and example-specific suggestions before the optimizer selects the final bounded update.

3.4 Bounded Text Updates

The learning-rate analogue in SKILLOPT is the edit budget L_t : the maximum number of skill edits applied at step t . After aggregation, the optimizer model ranks the merged edit pool by expected utility and clips it to the top L_t edits. This is the key difference from ad hoc prompt rewriting. Unbounded rewrites can erase useful rules, introduce incompatible instructions, or overfit to a local failure; bounded updates preserve continuity while still allowing the skill to acquire new procedures. SKILLOPT supports constant, linear, cosine, and autonomous schedules. The default cosine schedule starts with larger edits and decays toward smaller consolidation steps.

The selected edits produce a candidate skill. In patch mode, edits are localized operations such as append, insert, replace, and delete; in rewrite mode, selected suggestions condition a full skill rewrite. Step-level edits cannot overwrite the protected slow-update field, so fast local changes and slower epoch-wise consolidation remain separated.

3.5 Validation Gate and Rejected-Edit Buffer

Every candidate skill is evaluated on D_{sel} with the same frozen target model and harness. If it improves over the current selection score, it becomes the new current skill; if it also exceeds the best score so far, it becomes `best_skill.md`. Otherwise it is rejected. This gate turns reflection into propose-and-test optimization rather than unconditional self-editing, which is crucial because plausible textual diagnoses can still hurt the actual target model.

Rejected updates are still useful. The optimizer records an epoch-local buffer containing observed failure patterns and, for rejected steps, the edits that were tried and the score drop they caused. Later reflection calls in the same epoch receive this buffer, so the optimizer model can avoid repeating failed edits and focus on unresolved failures. This gives the loop negative feedback during training without adding inference-time cost.

3.6 Epoch-Wise Slow/Meta Update

Fast updates learn from the current batch; the epoch-wise slow/meta update learns from adjacent epochs. At the end of an epoch, SKILLOPT samples the same training items under the previous epoch’s skill and the current skill, then groups them into improvements, regressions, persistent failures, and stable successes. The optimizer model writes a concise longitudinal guidance block into a protected slow-update field, and this candidate is still passed through the validation gate. Thus slow update captures durable domain lessons while preserving the same safety check as step-level edits.

The meta skill is optimizer-side only. It summarizes which edit patterns helped, which were rejected, and which failures persisted across epochs. This meta guidance is prepended to future optimizer prompts for reflection, merging, and ranking, but it is not shipped with the target model. The advantage is separation of concerns: the deployed skill remains compact and portable, while training benefits from a richer record of the editing process.

3.7 Harness-Agnostic Deployment

SKILLOPT is harness-agnostic through a lightweight adapter interface, matching the broader trend toward agents embedded in tool-use and software-execution environments [1, 2, 4]. An adapter constructs train/evaluation batches, injects the current skill into the agent context, runs the native harness, and returns scored trajectories. The same optimizer therefore works for direct QA, spreadsheet execution, document reasoning, multimodal QA, embodied environments, and Codex-style or Claude Code-style execution loops. This is the main practical advantage of treating skills as the adaptation layer: a stronger optimizer model can train a reusable skill artifact offline, and the resulting `best_skill.md` can then be deployed or tested across target models, harnesses, and nearby benchmarks without changing model weights.

4 Experiments

We evaluate SKILLOPT as a text-space optimizer for frozen agents: the target model executes each task with the current skill, while an offline optimizer edits that skill from rollout evidence. The experiments answer four questions. (i) Do optimized skills improve over no-skill, human-skill, one-shot LLM-skill, prompt-optimization (TextGrad, GEPA), and skill-evolution (Trace2Skill, EvoSkill) baselines? (ii) Does the same loop work across direct chat, Codex, and Claude Code harnesses, and across seven target models from frontier-scale GPT to small Qwen? (iii) Which optimizer controls matter? (iv) What do the learned skills look like, and at what cost?

Setting. We report each benchmark’s native hard score or exact-match accuracy on held-out test splits across SearchQA [29], SpreadsheetBench [30], OfficeQA [31], DocVQA [32], LiveMathematicianBench [33] (abbreviated LiveMath in tables), and ALFWorld [34], using two model families:

Model	Skill source	SearchQA	Spreadsheet	OfficeQA	DocVQA	LiveMath	ALFWorld
No harness / direct chat							
GPT-5.5	No skill	77.7	41.8	33.1	78.8	37.6	83.6
	Human skill	81.8 _{+4.1}	72.9 _{+31.1}	66.9 _{+33.8}	90.1 _{+11.3}	38.4 _{+0.8}	91.8 _{+8.2}
	LLM skill	80.9 _{+3.2}	43.2 _{+1.4}	51.7 _{+18.6}	89.6 _{+10.8}	40.0 _{+2.4}	93.3 _{+9.7}
	Trace2Skill	82.4 _{+4.7}	49.6 _{+7.8}	65.7 _{+32.6}	90.6 _{+11.8}	52.0 _{+14.4}	87.3 _{+3.7}
	TextGrad	81.4 _{+3.7}	41.1 _{-0.7}	42.0 _{+8.9}	87.2 _{+8.4}	49.2 _{+11.6}	82.8 _{-0.8}
	GEPA	84.8 _{+7.1}	73.6 _{+31.8}	63.9 _{+30.8}	89.1 _{+10.3}	43.2 _{+5.6}	85.8 _{+2.2}
	SkillOpt	87.3_{+9.6}	80.7_{+38.9}	72.1_{+39.0}	91.2_{+12.4}	66.9_{+29.3}	95.5_{+11.9}
GPT-5.4	No skill	76.9	41.4	50.0	77.6	36.8	75.4
	Human skill	80.1 _{+3.2}	59.3 _{+17.9}	59.3 _{+9.3}	88.5 _{+10.9}	41.6 _{+4.8}	74.6 _{-0.8}
	LLM skill	79.4 _{+2.5}	46.1 _{+4.7}	20.4 _{-29.6}	90.4 _{+12.8}	36.8 _{+0.0}	83.6 _{+8.2}
	Trace2Skill	81.2 _{+4.3}	53.2 _{+11.8}	51.2 _{+1.2}	89.3 _{+11.7}	40.0 _{+3.2}	73.1 _{-2.3}
	TextGrad	82.5 _{+5.6}	38.6 _{-2.8}	54.7 _{+4.7}	85.3 _{+7.7}	36.6 _{-0.2}	78.4 _{+3.0}
	GEPA	82.4 _{+5.5}	61.1 _{+19.7}	60.4 _{+10.4}	88.3 _{+10.7}	41.6 _{+4.8}	74.6 _{-0.8}
	SkillOpt	83.1_{+6.2}	62.5_{+21.1}	62.8_{+12.8}	91.2_{+13.6}	44.0_{+7.2}	91.0_{+15.6}
GPT-5.4-mini	No skill	75.9	36.1	22.1	71.4	14.7	73.1
	Human skill	77.2 _{+1.3}	42.9 _{+6.8}	45.9 _{+23.8}	85.0 _{+13.6}	28.8 _{+14.1}	56.7 _{-16.4}
	LLM skill	76.1 _{+0.2}	36.8 _{+0.7}	36.6 _{+14.5}	86.4 _{+15.0}	28.0 _{+13.3}	65.7 _{-7.4}
	Trace2Skill	78.6 _{+2.7}	40.7 _{+4.6}	20.9 _{-1.2}	88.5 _{+17.1}	32.8 _{+18.1}	82.8 _{+9.7}
	TextGrad	77.5 _{+1.6}	38.2 _{+2.1}	30.0 _{+7.9}	84.0 _{+12.6}	27.2 _{+12.5}	70.9 _{-2.2}
	GEPA	79.4 _{+3.5}	42.5 _{+6.4}	45.3 _{+23.2}	83.7 _{+12.3}	27.2 _{+12.5}	81.3 _{+8.2}
	SkillOpt	80.2_{+4.3}	47.5_{+11.4}	48.8_{+26.7}	90.9_{+19.5}	32.8_{+18.1}	85.8_{+12.7}
GPT-5.4-nano	No skill	55.8	23.5	16.3	30.8	23.2	34.3
	Human skill	66.1 _{+10.3}	41.8 _{+18.3}	46.5 _{+30.2}	73.5 _{+42.7}	16.8 _{-6.4}	29.9 _{-4.4}
	LLM skill	62.4 _{+6.6}	33.6 _{+10.1}	12.8 _{-3.5}	71.7 _{+40.9}	20.0 _{-3.2}	65.7 _{+31.4}
	Trace2Skill	69.9 _{+14.1}	35.4 _{+11.9}	16.3 _{+0.0}	77.3 _{+46.5}	25.6 _{+2.4}	58.2 _{+23.9}
	TextGrad	73.4 _{+17.6}	32.5 _{+9.0}	38.7 _{+22.4}	68.3 _{+37.5}	20.8 _{-2.4}	42.5 _{+8.2}
	GEPA	73.2 _{+17.4}	37.2 _{+13.7}	45.0 _{+28.7}	71.2 _{+40.4}	24.8 _{+1.6}	68.7 _{+34.4}
	SkillOpt	74.8_{+19.0}	42.5_{+19.0}	50.0_{+33.7}	80.2_{+49.4}	27.2_{+4.0}	69.4_{+35.1}
GPT-5.2	No skill	71.9	38.2	34.9	73.1	20.8	68.7
	Human skill	75.8 _{+3.9}	48.2 _{+10.0}	55.2 _{+20.3}	89.0 _{+15.9}	23.2 _{+2.4}	56.7 _{-12.0}
	LLM skill	73.7 _{+1.8}	38.6 _{+0.4}	14.0 _{-20.9}	87.7 _{+14.6}	25.6 _{+4.8}	68.7 _{+0.0}
	Trace2Skill	79.2 _{+7.3}	43.9 _{+5.7}	43.0 _{+8.1}	87.7 _{+14.6}	28.8 _{+8.0}	76.9 _{+8.2}
	TextGrad	82.6 _{+10.7}	32.1 _{-6.1}	51.1 _{+16.2}	87.2 _{+14.1}	22.4 _{+1.6}	76.1 _{+7.4}
	GEPA	82.7 _{+10.8}	54.3 _{+16.1}	53.4 _{+18.5}	89.5 _{+16.4}	28.8 _{+8.0}	82.8 _{+14.1}
	SkillOpt	83.1_{+11.2}	57.1_{+18.9}	56.4_{+21.5}	89.6_{+16.5}	36.0_{+15.2}	85.1_{+16.4}
Qwen3.5-4B	No skill	68.1	9.3	14.5	86.9	22.4	30.6
	Human skill	66.3 _{-1.8}	16.4 _{+7.1}	22.7 _{+8.2}	87.8 _{+0.9}	18.4 _{-4.0}	28.4 _{-2.2}
	LLM skill	65.0 _{-3.1}	10.4 _{+1.1}	20.9 _{+6.4}	88.0 _{+1.1}	28.8 _{+6.4}	55.2 _{+24.6}
	Trace2Skill	68.5 _{+0.4}	19.3 _{+10.0}	16.3 _{+1.8}	88.0 _{+1.1}	27.2 _{+4.8}	64.9 _{+34.3}
	TextGrad	60.7 _{-7.4}	13.9 _{+4.6}	20.9 _{+6.4}	85.6 _{-1.3}	10.6 _{-11.8}	53.7 _{+23.1}
	GEPA	68.6 _{+0.5}	16.9 _{+7.6}	22.7 _{+8.2}	85.1 _{-1.8}	28.8 _{+6.4}	60.4 _{+29.8}
	SkillOpt	71.2_{+3.1}	23.9_{+14.6}	29.7_{+15.2}	89.0_{+2.1}	52.0_{+29.6}	81.3_{+50.7}
Qwen3.6-35B-A3B	No skill	72.7	38.2	45.9	87.6	31.2	59.7
	Human skill	74.1 _{+1.4}	44.3 _{+6.1}	41.9 _{-4.0}	88.4 _{+0.8}	29.6 _{-1.6}	44.8 _{-14.9}
	LLM skill	72.6 _{-0.1}	42.9 _{+4.7}	46.5 _{+0.6}	88.4 _{+0.8}	24.8 _{-6.4}	60.4 _{+0.7}
	Trace2Skill	75.4 _{+2.7}	33.2 _{-5.0}	32.0 _{-13.9}	90.4 _{+2.8}	29.6 _{-1.6}	70.9 _{+11.2}
	TextGrad	76.4 _{+3.7}	22.9 _{-15.3}	33.7 _{-12.2}	84.5 _{-3.1}	7.2 _{-24.0}	67.9 _{+8.2}
	GEPA	75.8 _{+3.1}	45.4 _{+7.2}	43.6 _{-2.3}	88.0 _{+0.4}	31.2 _{+0.0}	73.9 _{+14.2}
	SkillOpt	80.3_{+7.6}	47.5_{+9.3}	47.1_{+1.2}	91.4_{+3.8}	41.6_{+10.4}	82.1_{+22.4}
Codex harness							
GPT-5.5	No skill	81.8	27.5	38.3	87.2	35.2	—
	Human skill	84.1 _{+2.3}	50.7 _{+23.2}	40.0 _{+1.7}	88.8 _{+1.6}	48.8 _{+13.6}	—
	LLM skill	83.4 _{+1.6}	25.0 _{-2.5}	34.3 _{-4.0}	89.8 _{+2.6}	45.6 _{+10.4}	—
	EvoSkill	61.4 _{-20.4}	67.5 _{+40.0}	42.4 _{+4.1}	89.3 _{+2.1}	63.2 _{+28.0}	—
	SkillOpt	87.3_{+5.5}	85.0_{+57.5}	51.1_{+12.8}	92.2_{+5.0}	78.4_{+43.2}	—
Claude Code harness							
GPT-5.5	No skill	81.9	22.1	57.6	86.6	40.8	—
	Human skill	83.7 _{+1.8}	37.1 _{+15.0}	66.3 _{+8.7}	88.0 _{+1.4}	44.0 _{+3.2}	—
	LLM skill	82.4 _{+0.5}	37.1 _{+15.0}	56.4 _{-1.2}	89.6 _{+3.0}	44.8 _{+4.0}	—
	EvoSkill	84.0 _{+2.1}	75.0 _{+52.9}	70.3 _{+12.7}	87.2 _{+0.6}	52.0 _{+11.2}	—
	SkillOpt	85.9_{+4.0}	80.4_{+58.3}	71.5_{+13.9}	90.1_{+3.5}	56.5_{+15.7}	—

Table 1 Main results on held-out test splits. Scores are percentages; within each model-harness block, bold marks the best measured entry and underlining marks the second-best entry for each benchmark. Blue cells denote SKILLOPT, and small green/red subscripts show the absolute change relative to the *No skill* row of the same model in the same harness. We omit ALFWorld under Codex and Claude Code harnesses because ALFWorld requires persistent embodied-environment interaction. SKILLOPT is the best-or-tied entry on every measured cell of the table, with positive gains over the no-skill baseline throughout.

(a) Training set size				(b) Mini-batchsize				(c) Batchsize			
Setting	SearchQA	Spreadsheet	LiveMath	Setting	SearchQA	Spreadsheet	LiveMath	Setting	SearchQA	Spreadsheet	LiveMath
1 example	81.0	47.5	59.1	1	85.9	75.4	60.5	8	85.1	76.8	58.1
20% train	84.1	69.0	65.9	2	86.3	77.1	54.8	24	86.4	77.1	62.9
40% train	86.1	73.5	64.8	4	86.9	75.4	64.5	40	87.1	77.5	61.3
80% train	86.1	77.6	67.0	8	87.1	77.5	61.3	56	86.5	76.8	56.5
100% train	84.1	78.0	70.5	16	87.0	77.9	61.3	full epoch	87.2	75.0	53.2
				32	86.9	77.5	58.9				
(d) Learning rate				(e) Learning-rate scheduler				(f) Slow-update samples			
Setting	SearchQA	Spreadsheet	LiveMath	Setting	SearchQA	Spreadsheet	LiveMath	Setting	SearchQA	Spreadsheet	LiveMath
lr=1	85.5	77.5	62.1	constant	87.3	80.7	62.1	5	86.8	76.4	64.5
lr=2	86.7	77.5	60.5	cosine	87.1	77.5	61.3	10	86.4	74.3	65.3
lr=4	86.5	78.2	56.5	linear	87.2	72.9	62.9	20	87.1	77.5	61.3
lr=8	87.0	73.6	66.9					40	86.9	75.4	54.8
lr=16	86.8	78.2	65.3								

Table 2 Hyperparameter analysis for the text optimizer. Each panel changes one scalar or scheduling factor from the default setting unless noted. Panel (a) fixes the split to 4:1:5 train/selection/test; the 1-example, 20%, 40%, and 80% rows use subsets of the training partition, and the 100% row reuses the completed 4:1:5 split-ratio run. Panel (b) sweeps the reflection mini-batchsize B_m ; panel (c) sweeps the rollout batchsize B .

Component	Setting	SearchQA	SpreadsheetBench	LiveMath
Learning-rate form	lr=4 (default)	87.1	77.5	61.3
	dynamic lr	85.8	71.8	54.0
	without lr	84.6	75.7	57.3
Rejected buffer	with rejected buffer	87.1	77.5	61.3
	without rejected buffer	85.5	72.9	58.9
Slow/meta update	meta skill and slow update	87.1	77.5	61.3
	without meta skill	85.1	75.7	58.1
	without meta skill and slow update	86.3	55.0	59.7

Table 3 Component ablations for learning-rate form, rejected buffer, and epoch-wise slow/meta update. Light-blue rows mark the default setting within each component group; the learning-rate group uses the default lr=4 setting. Bold values mark the best measured result within that group and benchmark. The without-rejected-buffer row uses the matched no-buffer ablation setting.

GPT [35] and Qwen [36, 37]. The benchmark suite is intentionally diverse—it spans single-round QA (SearchQA, DocVQA, LiveMathematicianBench MCQ), multi-turn tool loops with up to 24 tool calls (OfficeQA), multi-round codegen with up to 30 turns and a real `openpyxl/pandas` runtime (SpreadsheetBench, default `mode=multi`), and persistent embodied interaction with up to 50 steps per episode (ALFWorld). Dataset-backed runs use deterministic train/selection/test splits derived from the same dataset seed (`split_seed = 42`); the selection split is used *only* to accept or reject candidate skill edits, and all reported scores are computed on the disjoint held-out test split. The reported numbers thus measure generalization, not validation-set fit.

Default optimizer hyperparameters. Unless noted, SKILLOPT uses four epochs, rollout batch size 40 per step, reflection minibatch size 8 (with 16 analyst workers running reflections in parallel and a merge batch size of 8), textual learning rate $L_t = 4$ with cosine decay (floor $L_t = 2$, configurable schedules: constant, linear, cosine, autonomous), held-out validation gating (strictly greater than the current selection score—ties are rejected), slow update with 20 sampled tasks per epoch comparing previous-epoch and current-epoch skill, an optimizer-side meta skill that summarizes accepted/rejected patterns into teacher-only guidance, the `patch` edit mode (the alternative is `rewrite_from_suggestions`), and an optional rejected-edit buffer of recent failed proposals. Teacher reflection is allowed up to three refinement rounds per minibatch. Both teacher and student calls default to a `medium` reasoning effort. For benchmarks with tightly bounded training pools (LiveMathematicianBench: 35 training items per epoch with rollout batch 200; ALFWorld: 39 training tasks with 140 selection and 134 test environments), per-benchmark configs scale the batch sizes accordingly while keeping the same gate, scheduler, and slow/meta-update machinery. Additional benchmark, baseline, and optimizer-protocol details are in Appendix C.

(a) Cross-model transfer					
Source model	Target model	Benchmark	Baseline	Direct	Transferred
GPT-5.4	GPT-5.4	SpreadsheetBench	41.4	62.5	52.1+10.7
	GPT-5.4-mini		36.1	47.5	45.5+9.4
	GPT-5.4-nano		23.5	42.5	26.5+3.0
GPT-5.4	GPT-5.4	LiveMath	36.8	44.0	47.2+10.4
	GPT-5.4-mini		14.7	32.8	19.2+4.5
	GPT-5.4-nano		23.2	27.2	28.8+5.6
(b) Cross-harness transfer					
Source harness	Target harness	Benchmark	Baseline	Direct	Transferred
Codex	Claude Code	LiveMath	40.8	56.5	42.4+1.6
Claude Code	Codex		35.2	78.4	48.0+12.8
Codex	Claude Code	SpreadsheetBench	22.1	80.4	81.8+59.7
Claude Code	Codex		27.5	85.0	71.1+43.6
(c) Cross-benchmark transfer					
Source benchmark	Target benchmark	Model	Baseline	Direct	Transferred
OlympiadBench	Omni-MATH	GPT-5.4	56.6	–	60.3+3.7
		GPT-5.4-mini	34.8	–	36.6+1.8
		GPT-5.4-nano	38.8	–	40.1+1.3

Table 4 Transfer of optimized skills across three axes. **(a) Cross-model:** a skill optimized for the source model is deployed on the target model. **(b) Cross-harness:** a skill trained inside the source harness is evaluated inside the target harness, all on GPT-5.5. **(c) Cross-benchmark:** the source benchmark skill is evaluated on the target benchmark across three target models. **Baseline** is the target’s no-skill score, **Direct** is the in-domain SKILLOPT score, and **Transferred** applies the source skill without further optimization. Subscripts show the change over the target baseline. Every row in (a)–(c) is a positive transfer (no row falls below the target’s no-skill baseline).

Harnesses. Direct chat invokes the target model through a single chat completion call with the skill prepended to the system prompt. The Codex harness drives the target through the `codex` CLI in a workspace-write sandbox [38]; SKILLOPT renders the current skill to a per-task `SKILL.md` alongside task files and reads back a compact execution trace (`codex_trace_summary.txt`) that is included in the teacher reflection context, so the optimizer learns from *what the agent actually did*, not just its final answer. The Claude Code harness mirrors the same workspace contract through the `claude` CLI [39]. All three modes consume the same `best_skill.md` file format, which is what enables the cross-harness transfer experiments in Section 4.3.

Baselines. We compare against seven baselines that span the no-adaptation, hand-written, one-shot, and learning families: *no skill* (frozen target model run with the benchmark’s default system prompt), *human skill* (an expert-written skill document curated per benchmark), *one-shot LLM skill* (a single skill generated from a high-level task description by GPT-5.5 and never updated), *Trace2Skill* [9] (trajectory-level skill distillation), *TextGrad* [40] (gradient-style natural-language prompt optimization), *GEPA* [13] (Pareto reflective prompt evolution), and the harness-side competitor *EvoSkill* [10] (skill-folder evolution under failure analysis). All baselines use the same target model, the same held-out test split, and the same scorer for every benchmark, so the comparison isolates the choice of adaptation procedure rather than secondary factors such as prompt template or scoring pipeline.

4.1 Main Results

Table 1 is the main result matrix. Counting every (target model, benchmark, harness) cell as one comparison and the strongest of the no-skill, human-skill, LLM-skill, Trace2Skill, TextGrad, GEPA, and EvoSkill baselines as the per-cell competition, SKILLOPT wins or matches the best

measured result on 52 of 52 evaluated cells. This dominance is uniform across model scales: SKILLOPT is best on every benchmark for GPT-5.5, GPT-5.4, GPT-5.4-mini, GPT-5.4-nano, GPT-5.2, Qwen3.5-4B, and Qwen3.6-35B-A3B in direct chat, and for GPT-5.5 under both Codex and Claude Code harnesses.

The size of the gains is also unusually large for a no-weight-update method. On GPT-5.5 direct chat, the six-benchmark average rises from 58.8 (no skill) to 82.3 (SKILLOPT), a +23.5 point absolute improvement, while the best per-cell baseline averages only 76.9, leaving SKILLOPT +5.4 points clear of an oracle baseline that picks the best of six competing methods per cell. Per-benchmark deltas over no skill range from +9.6 on SearchQA, where the no-skill model is already near ceiling, to +38.9 on SpreadsheetBench and +39.0 on OfficeQA, where strict procedural and answer-format requirements expose the limits of zero-shot frontier models. Procedural benchmarks see the largest improvements: SpreadsheetBench 41.8→80.7, OfficeQA 33.1→72.1, and LiveMathematicianBench 37.6→66.9 on GPT-5.5; SpreadsheetBench 9.3→23.9 ($\times 2.6$) on Qwen3.5-4B; and ALFWorld 34.3→69.4 ($\times 2.0$) on GPT-5.4-nano.

The improvement is not specific to frontier-scale targets. Averaged over the six benchmarks, SKILLOPT lifts GPT-5.4 by +12.7 points, GPT-5.4-mini by +15.4, GPT-5.4-nano by +26.7, GPT-5.2 by +16.6, Qwen3.5-4B by +19.2, and Qwen3.6-35B-A3B by +9.1, for an average improvement of approximately +17.6 points per model. Small and weak target models benefit the most in relative terms (e.g. GPT-5.4-nano nearly doubles on DocVQA and triples on ALFWorld), which is consistent with the view that a compact skill artifact can supply procedural knowledge that small models do not yet hold in weights.

The same optimization interface is also effective under tool-backed execution. On the Codex harness, SKILLOPT is best on all five evaluated benchmarks for GPT-5.5, with average gain +24.8 points over no skill and +14.0 over the next-best baseline (EvoSkill). On the Claude Code harness, it is best on all five benchmarks for GPT-5.5, with average gain +19.1 over no skill and +3.2 over EvoSkill, while EvoSkill itself already lifts the five-benchmark average from 57.8 to 73.7. The two ALFWorld cells under harness rows are left blank because ALFWorld requires persistent embodied-environment interaction that is not represented in the standard Codex / Claude Code adapters; we therefore report harness results on search, spreadsheets, document QA, multimodal QA, and math.

Taken together, the table supports a strong empirical claim: across direct chat and two tool-execution harnesses, across seven target models, and on procedural and factual benchmarks alike, optimizing a single compact skill artifact under bounded text-space training is the strongest no-weight-update adaptation strategy among the baselines we consider. The main gains come from feedback-driven skill editing rather than from a better one-shot prompt: human and LLM skills can help when prior instructions happen to match the benchmark, but they cannot correct failures after observing rollouts; Trace2Skill mines trajectory lessons without a held-out gate; TextGrad and GEPA optimize prompts but not a persistent skill artifact; and EvoSkill, the strongest harness-side competitor, lacks both bounded textual learning rates and rejected-edit memory. These comparisons support the central design choice—keep the target model, harness, and evaluator fixed, and optimize only the reusable skill artifact.

Alternative explanations. The per-cell baselines clarify what drives the gains. The effect is not simply prompt length: human skills are already 145–516 tokens long and often exceed the one-shot LLM skill, yet they are beaten in every direct-chat model row while the learned artifacts remain compact (Table 6). It is also not only optimizer capacity: SKILLOPT leads every baseline even for GPT-5.4-nano, and the optimizer-strength analysis in Table 5 shows that a target-matched optimizer recovers much of the gain. Finally, the harness results show that the method is not just exploiting one skill format: EvoSkill already improves the Codex SpreadsheetBench cell from 27.5 to 67.5, but SKILLOPT adds another +17.5 points (67.5→85.0). The gains are largest on

procedural benchmarks, where reusable rules about tool use and output formatting matter most, but they also appear on factual and multimodal benchmarks.

Headline numbers in one place. For convenience, the headline aggregates over Table 1 are: (i) 52/52 cells best or tied-best; (ii) average per-model improvement $\approx +17.6$ points across the seven direct-chat target models; (iii) average GPT-5.5 improvement of +23.5 (direct chat), +24.8 (Codex), +19.1 (Claude Code) over no skill; (iv) GPT-5.5 oracle-baseline gap of +5.4 points (direct chat) computed as the difference between SKILLOPT’s six-benchmark average (82.3) and an oracle that picks the best of six competing methods *per cell* (76.9). The remainder of this section unpacks why these gains appear (Section 4.2), how stable and transferable they are (Section 4.3), and what the learned artifact looks like (Section 4.4).

4.2 Ablations

Table 2, Figure 3, and Table 3 test the design choices in the optimizer using GPT-5.5 as both the target and the optimizer. The overall message is that SKILLOPT benefits from sufficient evidence, a bounded textual learning rate, rejected-edit feedback, and epoch-wise slow/meta update. SearchQA has limited headroom and is therefore stable across many settings (most cells fluctuate inside a ± 1.5 point band), while SpreadsheetBench and LiveMathematicianBench expose the trade-off between learning useful procedures and over-editing the skill.

Evidence and batch sizes (panels a, b, c). Panel (a) shows that procedural benchmarks reward more training evidence: SpreadsheetBench climbs 47.5 $\rightarrow$ 78.0 and LiveMathematicianBench climbs 59.1 $\rightarrow$ 70.5 as the optimizer sees 1 $\rightarrow$ 100% of the training partition, while SearchQA saturates at roughly 84–86 after 20% already. Panel (b) shows the same robustness in the other direction: varying the reflection mini-batchsize from 1 to 32 keeps SearchQA inside 85.9–87.1 and SpreadsheetBench inside 75.4–77.9, with the default $B_m=8$ at or near the top on all three benchmarks. Panel (c) is equally flat in the rollout-batchsize dimension—moving from $B=8$ to a full epoch keeps SearchQA inside 85.1–87.2 and SpreadsheetBench inside 75.0–77.5. Together this means the headline gains are not the product of a fragile prompt-search batch size, but a genuine effect of having enough scored evidence per update.

Textual learning rate and schedule (panels d, e). Panels (d) and (e) directly compare bounded textual learning to looser settings. Sweeping $L_t \in \{1, 2, 4, 8, 16\}$ shows that small or moderate edit budgets are competitive throughout: $L_t=4$ achieves 86.5/78.2/56.5, the highest LiveMath score belongs to $L_t=8$ at 66.9, and the lowest score across all five settings is still only 85.5 on SearchQA. Panel (e) confirms this on the schedule axis: constant decay scores 87.3/80.7/62.1, cosine 87.1/77.5/61.3, and linear 87.2/72.9/62.9, so the bounded-update story does not depend on a single specific scheduler. The qualitative claim is what matters: any moderate, bounded edit budget already beats baselines that rewrite the skill without a budget (Table 3, “without lr” row, 84.6/75.7/57.3).

Epoch-wise slow/meta update (panel f, Table 3, Figure 3). The slow/meta update supplies longer-horizon guidance beyond the current rollout batch. Slow-update sampling (panel f) places the default at 20 examples per epoch (87.1, 77.5, and 61.3), with 5, 10, and 40 each within ± 2.7 points. In the matched default component row, removing the rejected-edit buffer lowers scores by 1.6, 4.6, and 2.4 points on SearchQA, SpreadsheetBench, and LiveMath, respectively, supporting its role as a stabilizer for the default loop rather than as an extra deployment-time mechanism. The slow/meta ablation rows are sharper: removing both meta skill and slow update drops SpreadsheetBench from 77.5 to 55.0 (−22.5 points), the largest degradation in the ablation suite. Figure 3 complements these numerical ablations: validation checkpoints track held-out test

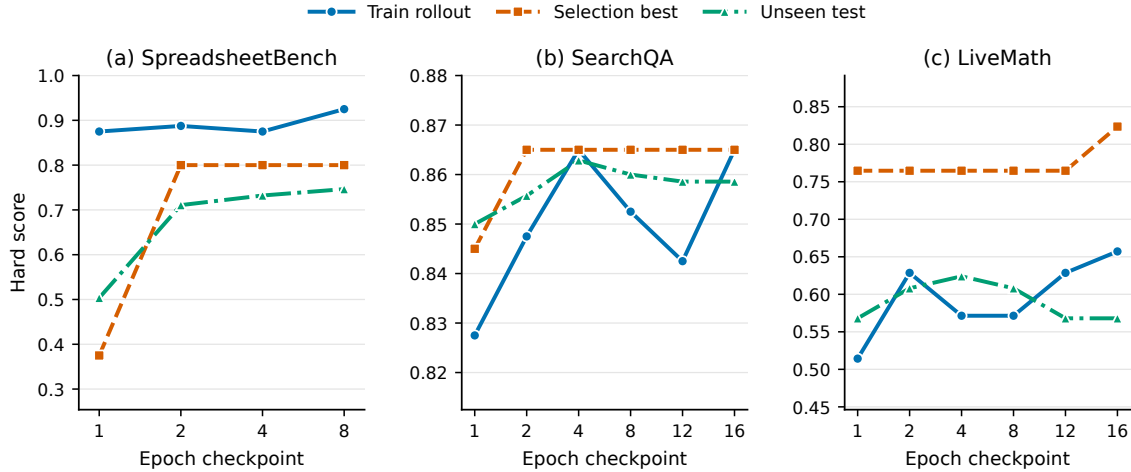


Figure 3 Performance trends across epoch checkpoints on three benchmarks: (a) SpreadsheetBench, (b) SearchQA, and (c) LiveMath. For each checkpoint, we report the training rollout score, the selection-best score on the validation set, and the final performance on the unseen test set. The results show how skill quality evolves during optimization and whether the checkpoint preferred by validation selection aligns with the checkpoint that yields the best generalization to the test set.

performance across epochs, confirming that the gate tends to select skills that generalize rather than skills that only fit the selection split.

Gate strictness and edit observability. The validation gate is intentionally strict: a candidate skill is accepted only when its selection-split score is *strictly greater than* the current selection score, so ties are rejected and the deployed skill never silently drifts. This conservative criterion makes rejected edits informative negative feedback rather than hidden state. Operationally, every step also records an `edit_apply_report.json` containing per-edit accept/skip status, so the source of every change to `best_skill.md` is recoverable after the fact. The epoch-wise slow/meta update writes into a markup-fenced protected region of the skill document that step-level edits cannot overwrite, separating the fast intra-epoch update from the slower cross-epoch consolidation; the optimizer-side meta skill lives only in the teacher’s reflection context and is never shipped with the deployed artifact. These implementation choices explain why removing both meta skill and slow update is especially damaging on SpreadsheetBench: it removes the long-horizon evidence stream and the protected-region contract that keeps local edits from overwriting durable procedural lessons.

Overall, the ablations show that the gains are relatively insensitive to the exact rollout batch, reflection minibatch, or learning-rate schedule, but much more sensitive to the presence of bounded text-space learning, validation gating, rejected-edit feedback, and epoch-wise slow/meta update—the design choices that make skill editing behave like a controlled training loop.

4.3 Analysis and Transfer

Tables 4–4 ask whether an optimized skill behaves like a reusable artifact rather than a task-specific prompt. We test three shifts: deploying a skill across model scales (Table 4), moving it across execution harnesses (Table 4), and applying it to nearby math benchmarks, including OlympiadBench [41] and Omni-MATH [14] (Table 4). Table 5 then asks how much of the gain depends on optimizer capacity by replacing the frontier optimizer with a target-matched one of the same scale as the deployed model.

Cross-model transfer. Table 4(a) is uniformly positive: every cross-model row shows a gain over the target’s no-skill baseline. SpreadsheetBench skills trained with GPT-5.4 transfer to GPT-5.4

(+10.7), GPT-5.4-mini (+9.4), and GPT-5.4-nano (+3.0); LiveMath skills transfer to GPT-5.4 (+10.4), GPT-5.4-mini (+4.5), and GPT-5.4-nano (+5.6). On two of the six rows the transferred skill *surpasses* the in-domain SKILLOPT reference (LiveMath GPT-5.4: 47.2 transferred vs. 44.0 in-domain; LiveMath GPT-5.4-nano: 28.8 transferred vs. 27.2 in-domain), suggesting that some learned procedures are target-model agnostic. The remaining rows still recover a useful fraction of the in-domain gain—e.g. SpreadsheetBench GPT-5.4 retains roughly half of the in-domain gain (+10.7 of +21.1)—and no row falls below the target’s no-skill baseline.

Cross-harness transfer. The harness-shift rows in Table 4(b) are the clearest deployment signal. A SpreadsheetBench skill trained inside the Codex loop transfers to Claude Code with absolute gain +59.7 over the Claude Code no-skill baseline (22.1→81.8, slightly exceeding the in-domain Claude Code SKILLOPT reference of 80.4), and the symmetric Claude-Code→Codex transfer adds +43.6 on top of the Codex baseline (27.5→71.1). On LiveMath, the Codex→Claude Code transfer is smaller (+1.6 over a 40.8 baseline) but still positive, while the Claude-Code→Codex transfer adds +12.8 (35.2→48.0). Because the two harnesses expose different tool/file APIs and command surfaces, these positive transfers suggest that the learned rules are not only harness-specific command recipes. In SpreadsheetBench especially, the transferred skill appears to encode workbook-level procedures such as structure-first inspection, formula-aware verification, and static-value materialization, so the cost of optimizing a skill in one execution environment can be amortized across related deployment environments.

Cross-benchmark transfer. Cross-benchmark transfer is the strictest of the three shifts: source and target benchmarks share only the broad task family (math). On the OlympiadBench→OmniMATH direction reported in Table 4(c), the transferred skill is positive on all three model scales we evaluate, with gains of +3.7 on GPT-5.4, +1.8 on GPT-5.4-mini, and +1.3 on GPT-5.4-nano. These rows are smaller than the in-domain and cross-harness transfers—unsurprisingly, since they require the optimized skill to retain useful procedural knowledge after both the test instances and the answer-format conventions change—but they remain uniformly positive, supporting the intended interpretation that the optimized skill encodes reusable mathematical procedure rather than memorized benchmark-specific formatting.

Effect of optimizer strength. Because the optimizer in SKILLOPT runs only during the offline training loop and is never invoked at deployment, optimizer choice is a training-time lever: a stronger optimizer can improve the deployed skill without raising the inference cost of using that skill. The deployed artifact is still a static `best_skill.md` that calls only the target model. Table 5 quantifies this lever by running the same loop with two optimizer regimes—a strong frontier optimizer (GPT-5.5) and a target-matched optimizer that shares the target model—while holding the rollout batches, validation gate, bounded edit budget, rejected-edit buffer, and slow/meta update identical.

Two observations follow. *First*, the stronger optimizer produces larger absolute gains on every (benchmark, target) cell we test: GPT-5.4-nano lifts by +19.0 vs. +11.9 on SpreadsheetBench and +19.0 vs. +14.1 on SearchQA, and GPT-5.4-mini follows the same ordering (+11.4 vs. +7.1 on SpreadsheetBench, +4.3 vs. +2.4 on SearchQA). The bounded-edit, validation-gated loop is what makes this monotone: without the gate, a stronger optimizer could just as easily push larger but harmful rewrites. *Second*, the target-matched optimizer is far from collapsed—it recovers 56–74% of the strong-optimizer gain across the four cells, confirming that SKILLOPT is not a distillation pipeline from a stronger teacher into a weaker student: the optimization loop itself contributes substantial value on top of whatever the optimizer can already do. The practical implication is that a high-capacity frontier optimizer is the right default whenever it is available—it costs only

Benchmark	Target	Baseline	Strong optimizer (GPT-5.5)	Target-matched optimizer
SpreadsheetBench	GPT-5.4-mini	36.1	47.5 _{+11.4}	43.2 _{+7.1}
	GPT-5.4-nano	23.5	42.5 _{+19.0}	35.4 _{+11.9}
SearchQA	GPT-5.4-mini	75.9	80.2 _{+4.3}	78.3 _{+2.4}
	GPT-5.4-nano	55.8	74.8 _{+19.0}	69.9 _{+14.1}

Table 5 Effect of optimizer strength. Each (benchmark, target) pair is optimized either by a strong frontier optimizer (GPT-5.5, bolded) or by a target-matched optimizer that shares the target model; everything else in the SKILLOPT loop is held fixed. Gains over the target’s no-skill baseline are shown as small green subscripts; the same baseline is used for both optimizer settings within a row. The optimizer runs only during offline training, so the stronger-optimizer column adds zero cost at deployment.

Benchmark	Initial (tok)	Final (tok)	Edits	Train tokens	Cost / pt
SearchQA	16	857	4	213.8M	37.9M
SpreadsheetBench	224	1,995	4	21.4M	0.6M
OfficeQA	145	883	1	20.8M	1.1M
DocVQA	81	959	3	188.2M	46.4M
LiveMath	154	379	1	23.2M	3.6M
ALFWorld	516	1,321	2	59.3M	15.9M

Table 6 Cost and edit economy of the GPT-5.5 / GPT-5.5 (student / teacher) skill runs. Initial and final `best_skill.md` lengths are in tokens; **Edits** is the number of accepted bounded updates; **Cost / pt** is training tokens per absolute test-point gain. One representative learned rule per benchmark is shown in Figure 4.

training-time API calls and adds nothing to deployment—while the same loop remains effective if the budget forces a target-matched optimizer instead.

4.4 Learned Skills: Compactness, Cost, and Examples

A central premise of SKILLOPT is that the trainable object should remain a small, inspectable text document. Tables 1–5 demonstrate that the optimizer is effective; this subsection asks what its output actually looks like and what it costs. We characterize the learned artifact on three axes—compactness, edit economy, and cost-per-point—and then show one representative learned rule per benchmark to illustrate what kind of procedural knowledge survives the bounded-update loop.

Compactness. The final skills are uniformly small. Across the six benchmarks in Table 6, the final `best_skill.md` ranges from 379 tokens (LiveMathematicianBench) to 1,995 tokens (SpreadsheetBench), with a median of roughly 920 tokens. Even the longest learned skill is well below a typical system-prompt budget for modern frontier models, and the shortest one fits inside a single screen. The growth from initial to final skill is moderate ($\times 2.5$ to $\times 53$ depending on whether the initial skill was a one-liner or a paragraph), but the final size in absolute tokens stays small enough that a domain practitioner can read, audit, and edit the deployed artifact in minutes.

Edit economy. A second striking pattern is that the gains come from very few accepted edits. Across the six benchmarks, the number of edits actually committed to `best_skill.md` during optimization is between 1 and 4 (median 2.5). LiveMathematicianBench’s $+29.3$ point gain over no skill arises from a *single* accepted edit, and OfficeQA’s $+39.0$ point gain similarly arises from one accepted edit. This is direct evidence that the validation gate is doing real work: the optimizer model proposes many more edits per epoch, but only a handful pass the held-out check and survive into the deployed skill. The bulk of the optimizer’s text-space search is thus rejected, captured by the rejected-edit buffer (Section 3.5) for future use, and never reaches the target model. The deployed skill is correspondingly compact rather than the union of every reflection.

SearchQA. “Infer the expected answer type from clue wording, then choose the shortest canonical entity supported by co-occurring distinctive evidence.”
SpreadsheetBench. “Inspect workbook structure and formulas, then write evaluated static values across the full requested target range instead of relying on Excel recalculation.”
OfficeQA. “Treat oracle parsed pages as primary evidence, lock table/date/unit context, and output exactly the requested rounded value without extra labels.”
DocVQA. “For tables, forms, charts, and legends, first bind the question to the exact visual row/header/field, then copy only the aligned answer span.”
LiveMathematicianBench. “In strongest-statement MCQs, rank choices by theorem strength and prefer a justified stronger-result option over true but weaker corollaries.”
ALFWorld. “Keep a horizon-aware visited/frontier ledger, diversify search after repeated same-type failures, and avoid revisiting the destination until holding the target.”

Figure 4 Representative learned rules, one per benchmark, extracted from the final `best_skill.md` of the GPT-5.5 / GPT-5.5 runs in Table 6. Each rule is verbatim from the deployed skill. Notably, every rule is procedural rather than instance-specific, and several encode forms of discipline (answer formatting, evidence binding, search-frontier management) that frontier models do not apply zero-shot.

Cost per point of test-set gain. The training-token column quantifies the cost of operating the loop. Two regimes are visible. Procedural benchmarks where rollouts are short and cheap—SpreadsheetBench, OfficeQA, LiveMathematicianBench—reach 0.6–3.6M training tokens per absolute test-set point, even though the absolute gains on these benchmarks are the largest (e.g. +39.0 points on OfficeQA at 1.1M tokens / point, total 20.8M tokens). Benchmarks with longer trajectories or richer multimodal context—SearchQA (37.9M / pt) and DocVQA (46.4M / pt)—cost an order of magnitude more per point. The important deployment distinction is that this cost is paid once during skill training; after export, the optimized `best_skill.md` adds no optimizer calls, no weight updates, and only a compact text artifact to the target agent.

What does a learned skill actually say? Figure 4 reproduces one representative learned rule per benchmark, taken verbatim from the final `best_skill.md` of each case study in Table 6. Three observations stand out. First, the rules are *procedural* rather than *instance-specific*: none of them name a specific question, file, or entity. Second, they consistently encode the discipline that frontier models lack zero-shot: answer-format constraints (OfficeQA, LiveMathematicianBench), evidence binding to a specific visual region (DocVQA), workbook-structure-first reasoning (SpreadsheetBench), search-frontier discipline (ALFWorld), and canonical-entity choice (SearchQA). Third, they read like rules a thoughtful human practitioner would write after a day with the benchmark—except they are produced automatically by the optimizer and validated edit-by-edit on held-out data.

Implications. Together, the four observations above support a stronger version of the central claim. Compactness (< 2,000 tokens) and edit economy (1–4 accepted edits) mean the deployed artifact is interpretable. Cost-per-point (0.6M–46.4M tokens / point) shows that the training cost is measurable and paid before deployment. The shape of the learned rules—procedural, generalizable, and consistent with what a thoughtful human practitioner would write—is evidence that text-space optimization with bounded updates and validation gating discovers transferable procedural knowledge rather than merely overfitting to the training split. This complements the cross-model, cross-harness, and cross-benchmark transfer evidence in Section 4.3: the artifact transfers because many of the rules it encodes are intrinsically transferable.

4.5 Qualitative Skill Evolution

We inspect two representative runs to understand what the optimized skill actually learns. The ALFWorld case uses GPT-5.4-nano as the student and GPT-5.5 as the teacher, while the

SpreadsheetBench case uses GPT-5.5 as both the frozen student and optimizer model. In both cases, SKILLOPT does not replace the initial skill with an unrelated prompt. Instead, accepted edits add compact procedural constraints around recurring failure modes observed in rollout trajectories.

ALFWorld. The initial ALFWorld skill gives a generic household plan: search for the target object, pick it up, transform it if needed, and place it at the destination. The accepted edits make this plan more stateful and less loop-prone. The optimized skill learns exact object-name matching, so related objects such as mugs, cups, pans, and pots are not substituted for one another. It adds visited-location memory, so unvisited receptacles and surfaces are preferred over repeatedly checking likely but exhausted locations. It also adds destination memory, pick-two progress locks, and direct completion rules: once the agent can clean, heat, cool, place, or otherwise complete the next subgoal, it should take that admissible action instead of examining, closing, or verifying again. Qualitatively, the skill evolves from a general search-transform-place strategy into a finite-state execution policy with object identity, search memory, progress locks, and loop breakers. In this representative run, the selected skill improves ALFWorld held-out test performance from 49.3 to 74.6.

SpreadsheetBench. The initial SpreadsheetBench skill already instructs the agent to use Python spreadsheet libraries and preserve unrelated workbook content. The accepted edits turn this generic automation workflow into a workbook-forensics policy. The optimized skill learns to inspect the actual workbook rather than rely on previews, locate headers and target ranges across multiple sheets, normalize keys and cell types before lookup or aggregation, and preserve formatting during structural edits. It also adds a key rule for formula-style prompts: when the grader reads cell values, the agent should compute and write evaluated static values, even if the prompt mentions formulas such as INDEX/MATCH or XLOOKUP. Later edits further require filling complete target ranges, including currently blank result cells, keeping helper computations in Python rather than adding workbook artifacts, and reopening the saved workbook to check boundary rows and remaining blanks. In this representative run, the selected skill improves SpreadsheetBench held-out test performance from 40.4 to 78.9.

5 Conclusion

We presented SKILLOPT, a text-space optimizer that treats an external skill document as the trainable state for frozen LLM agents. By separating the target model that executes tasks from the optimizer that edits skills, and by using bounded edit budgets, minibatch reflection, held-out validation gates, rejected-edit buffers, and epoch-wise slow/meta update, SKILLOPT turns skill improvement into a controlled learning process rather than ad hoc prompt revision. Across six benchmarks, seven target models, and three execution modes, SKILLOPT is best or tied-best on 52 of 52 evaluated cells, lifts GPT-5.5 by +23.5 points on average over no skill in direct chat and by +24.8/ + 19.1 points under Codex and Claude Code harnesses, and beats the strongest per-cell baseline from human, LLM, Trace2Skill, TextGrad, GEPA, and EvoSkill skills by +5.4 points on average. Per-benchmark case studies show that these gains arise from compact (< 2,000 token), interpretable skill artifacts assembled from only 1–4 accepted edits, and that the deployed skills transfer across model scales, harnesses, and nearby benchmarks. These results suggest that compact natural-language skills can serve as a practical domain-adaptation layer for frontier agents, enabling reusable improvement without modifying model weights.

Outlook. SKILLOPT optimizes a single skill artifact for a single target domain; natural extensions include skill libraries that share infrastructure across domains, reuse of optimizer-side meta skills

across benchmarks, reward-free or preference-driven validation gates for open-ended tasks, and self-distillation of optimized skills back into the target model as a stepping stone toward weight-level adaptation. We hope that treating the skill itself as the trainable object—rather than as a side artifact of prompting—will let future work apply the full toolkit of optimization (learning rates, schedules, regularization, curricula, validation) to a part of the agent stack that has so far been hand-engineered.

References

- [1] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*, 2022.
- [2] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. *Advances in neural information processing systems*, 36: 68539–68551, 2023.
- [3] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291*, 2023.
- [4] John Yang, Carlos E Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. Swe-agent: Agent-computer interfaces enable automated software engineering. *Advances in Neural Information Processing Systems*, 37:50528–50652, 2024.
- [5] Chengrun Yang, Xuezhi Wang, Yifeng Lu, Hanxiao Liu, Quoc V Le, Denny Zhou, and Xinyun Chen. Large language models as optimizers. In *The Twelfth International Conference on Learning Representations*, 2023.
- [6] Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T Joshi, Hanna Moazam, et al. Dspy: Compiling declarative language model calls into self-improving pipelines. *arXiv preprint arXiv:2310.03714*, 2023.
- [7] Xiangyi Li, Wenbo Chen, Yimin Liu, Shenghan Zheng, Xiaokun Chen, Yifeng He, Yubo Li, Bingran You, Haotian Shen, Jiankai Sun, et al. Skillsbench: Benchmarking how well agent skills work across diverse tasks. *arXiv preprint arXiv:2602.12670*, 2026.
- [8] Yanna Jiang, Delong Li, Haiyu Deng, Baihe Ma, Xu Wang, Qin Wang, and Guangsheng Yu. Sok: Agentic skills—beyond tool use in llm agents. *arXiv preprint arXiv:2602.20867*, 2026.
- [9] Jingwei Ni, Yihao Liu, Xinpeng Liu, Yutao Sun, Mengyu Zhou, Pengyu Cheng, Dexin Wang, Xiaoxi Jiang, and Guanjin Jiang. Trace2skill: Distill trajectory-local lessons into transferable agent skills. *arXiv preprint arXiv:2603.25158*, 2026.
- [10] Salaheddin Alzubi, Noah Provenzano, Jaydon Bingham, Weiyuan Chen, and Tu Vu. Evoskill: Automated skill discovery for multi-agent systems. *arXiv preprint arXiv:2603.02766*, 2026.
- [11] Xingyan Liu, Xiyue Luo, Linyu Li, Ganghong Huang, Jianfeng Liu, and Honglin Qiao. Skillforge: Forging domain-specific, self-evolving agent skills in cloud technical support. *arXiv preprint arXiv:2604.08618*, 2026.

- [12] Shuaike Shen, Wenduo Cheng, Mingqian Ma, Alistair Turcan, Martin Jinze Zhang, and Jian Ma. Skillfoundry: Building self-evolving agent skill libraries from heterogeneous scientific resources. *arXiv preprint arXiv:2604.03964*, 2026.
- [13] Lakshya A Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziemis, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, Michael J Ryan, Meng Jiang, et al. Gepa: Reflective prompt evolution can outperform reinforcement learning. *arXiv preprint arXiv:2507.19457*, 2025.
- [14] Bofei Gao, Feifan Song, Zhe Yang, Zefan Cai, Yibo Miao, Qingxiu Dong, Lei Li, Chenghao Ma, Liang Chen, Runxin Xu, Zhengyang Tang, Benyou Wang, Daoguang Zan, Shanghaoran Quan, Ge Zhang, Lei Sha, Yichang Zhang, Xuancheng Ren, Tianyu Liu, and Baobao Chang. Omni-math: A universal olympiad level mathematic benchmark for large language models, 2024. URL <https://arxiv.org/abs/2410.07985>.
- [15] Weijia Song, Jiashu Yue, and Zhe Pang. Abstral: Automatic design of multi-agent systems through iterative refinement and topology optimization. *arXiv preprint arXiv:2603.22791*, 2026.
- [16] Yufei He, Juncheng Liu, Yue Liu, Yibo Li, Tri Cao, Zhiyuan Hu, Xinxing Xu, and Bryan Hooi. Evotest: Evolutionary test-time learning for self-improving agentic systems. *arXiv preprint arXiv:2510.13220*, 2025.
- [17] Yutao Yang, Junsong Li, Qianjun Pan, Bihao Zhan, Yuxuan Cai, Lin Du, Jie Zhou, Kai Chen, Qin Chen, Xin Li, et al. Autoskill: Experience-driven lifelong learning via skill self-evolution. *arXiv preprint arXiv:2603.01145*, 2026.
- [18] Chenxi Wang, Zhuoyun Yu, Xin Xie, Wuguannan Yao, Runnan Fang, Shuofei Qiao, Kexin Cao, Guozhou Zheng, Xiang Qi, Peng Zhang, et al. Skillx: Automatically constructing skill knowledge bases for agents. *arXiv preprint arXiv:2604.04804*, 2026.
- [19] Runnan Fang, Yuan Liang, Xiaobin Wang, Jialong Wu, Shuofei Qiao, Pengjun Xie, Fei Huang, Huajun Chen, and Ningyu Zhang. Memp: Exploring agent procedural memory. *arXiv preprint arXiv:2508.06433*, 2025.
- [20] Hanrong Zhang, Shicheng Fan, Henry Peng Zou, Yankai Chen, Zhenting Wang, Jiayu Zhou, Chengze Li, Wei-Chieh Huang, Yifei Yao, Kening Zheng, et al. Evoskills: Self-evolving agent skills via co-evolutionary verification. *arXiv preprint arXiv:2604.01687*, 2026.
- [21] Ziyu Ma, Shidong Yang, Yuxiang Ji, Xucong Wang, Yong Wang, Yiming Hu, Tongwen Huang, and Xiangxiang Chu. Skillclaw: Let skills evolve collectively with agentic evolver. *arXiv preprint arXiv:2604.08377*, 2026.
- [22] Peng Xia, Jianwen Chen, Hanyang Wang, Jiaqi Liu, Kaide Zeng, Yu Wang, Siwei Han, Yiyang Zhou, Xujiang Zhao, Haifeng Chen, et al. Skillrl: Evolving agents via recursive skill-augmented reinforcement learning. *arXiv preprint arXiv:2602.08234*, 2026.
- [23] Jiongxiao Wang, Qiaojing Yan, Yawei Wang, Yijun Tian, Soumya Smruti Mishra, Zhichao Xu, Megha Gandhi, Panpan Xu, and Lin Lee Cheong. Reinforcement learning for self-improving agent with skill library. *arXiv preprint arXiv:2512.17102*, 2025.
- [24] Libin Qiu, Zhirong Gao, Junfu Chen, Yuhang Ye, Weizhi Huang, Xiaobo Xue, Wenkai Qiu, and Shuo Tang. Autorefine: From trajectories to reusable expertise for continual llm agent refinement. *arXiv preprint arXiv:2601.22758*, 2026.

- [25] Qirui Mi, Zhijian Ma, Mengyue Yang, Haoxuan Li, Yisen Wang, Haifeng Zhang, and Jun Wang. Procmem: Learning reusable procedural memory from experience via non-parametric ppo for llm agents. *arXiv preprint arXiv:2602.01869*, 2026.
- [26] Rong Wu, Xiaoman Wang, Jianbiao Mei, Pinlong Cai, Daocheng Fu, Cheng Yang, Licheng Wen, Xueming Yang, Yufan Shen, Yuxin Wang, et al. Evolver: Self-evolving llm agents through an experience-driven lifecycle. *arXiv preprint arXiv:2510.16079*, 2025.
- [27] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *Advances in neural information processing systems*, 36:8634–8652, 2023.
- [28] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, et al. Self-refine: Iterative refinement with self-feedback. *Advances in neural information processing systems*, 36:46534–46594, 2023.
- [29] Matthew Dunn, Levent Sagun, Mike Higgins, V Ugur Guney, Volkan Cirik, and Kyunghyun Cho. Searchqa: A new q&a dataset augmented with context from a search engine. *arXiv preprint arXiv:1704.05179*, 2017.
- [30] Zeyao Ma, Bohan Zhang, Jing Zhang, Jifan Yu, Xiaokang Zhang, Xiaohan Zhang, Sijia Luo, Xi Wang, and Jie Tang. Spreadsheetbench: Towards challenging real world spreadsheet manipulation. *Advances in Neural Information Processing Systems*, 37:94871–94908, 2024.
- [31] Krista Opsahl-Ong, Arnav Singhvi, Jasmine Collins, Ivan Zhou, Cindy Wang, Ashutosh Baheti, Owen Oertell, Jacob Portes, Sam Havens, Erich Elsen, et al. Officeqa pro: An enterprise benchmark for end-to-end grounded reasoning. *arXiv preprint arXiv:2603.08655*, 2026.
- [32] Minesh Mathew, Dimosthenis Karatzas, and CV Jawahar. Docvqa: A dataset for vqa on document images. In *Proceedings of the IEEE/CVF winter conference on applications of computer vision*, pages 2200–2209, 2021.
- [33] Linyang He, Qiyao Yu, Hanze Dong, Baohao Liao, Xinxing Xu, Micah Goldblum, Jiang Bian, and Nima Mesgarani. Livemathematicianbench: A live benchmark for mathematician-level reasoning with proof sketches, 2026. URL <https://arxiv.org/abs/2604.01754>.
- [34] Mohit Shridhar, Xingdi Yuan, Marc-Alexandre Cote, Yonatan Bisk, Adam Trischler, and Matthew Hausknecht. {ALFW}orld: Aligning text and embodied environments for interactive learning. In *International Conference on Learning Representations*, 2021. URL <https://openreview.net/forum?id=0IOX0YcCdTn>.
- [35] OpenAI. Introducing GPT-5.4, March 2026. URL <https://openai.com/index/introducing-gpt-5-4/>.
- [36] Qwen Team. Qwen3.5: Towards native multimodal agents, February 2026. URL <https://qwen.ai/blog?id=qwen3.5>.
- [37] Qwen Team. Qwen3.6-35B-A3B: Agentic coding power, now open to all, April 2026. URL <https://qwen.ai/blog?id=qwen3.6-35b-a3b>.
- [38] OpenAI. Codex: A cloud-based software engineering agent, 2025. URL <https://openai.com/index/introducing-codex/>. Accessed: 2026-05-06.

- [39] Anthropic. Claude code: An ai coding agent system, 2025. URL <https://www.anthropic.com/claude-code>. Accessed: 2026-05-06.
- [40] Mert Yuksekgonul, Federico Bianchi, Joseph Boen, Sheng Liu, Zhi Huang, Carlos Guestrin, and James Zou. Textgrad: Automatic “differentiation” via text. *arXiv preprint arXiv:2406.07496*, 2024.
- [41] Chaoqun He, Renjie Luo, Yuzhuo Bai, Shengding Hu, Zhen Leng Thai, Junhao Shen, Jinyi Hu, Xu Han, Yujie Huang, Yuxiang Zhang, et al. Olympiadbench: A challenging benchmark for promoting agi with olympiad-level bilingual multimodal scientific problems. *arXiv preprint arXiv:2402.14008*, 2024.

A Additional Method Details and Optimizer Prompts

This appendix gives the executable details behind SKILLOPT. The optimization loop keeps the task-execution model fixed and trains only a text skill document. A separate optimizer model reads rollout evidence, proposes patch-style edits, merges and ranks the edits, and submits each candidate skill to a held-out selection gate. The task-execution model only receives the current skill and the benchmark task; it does not see the optimizer prompts below.

B Limitations

SKILLOPT studies skill optimization as a lightweight alternative to model-weight adaptation, but it still has several practical limitations. First, the optimization loop relies on scored trajectories and a held-out selection split, so it is most directly applicable when the target task has automatic verifiers, exact-match metrics, executable checks, or otherwise reliable feedback signals. For open-ended domains where success is subjective, multi-dimensional, or costly to judge, the validation gate may require stronger human or model-based evaluation. Second, although the deployed artifact is only a compact `best_skill.md`, training the skill requires additional rollout computation and calls to an optimizer model; this cost is amortized when the same skill is reused, but may be less attractive for one-off tasks. Third, SKILLOPT intentionally optimizes a single portable skill rather than growing a large skill library or changing model weights. This design improves deployment simplicity, but a single skill may be insufficient for highly heterogeneous domains that require many disjoint procedures. Finally, optimized skills can encode domain-specific heuristics from the training distribution, so careful held-out evaluation remains necessary before transferring them to substantially different models, harnesses, or task settings.

C Experimental Protocol Details

Benchmarks and metrics. We use each benchmark’s native evaluator and report hard success or exact-match accuracy on held-out test examples. SearchQA measures extractive question answering; SpreadsheetBench evaluates spreadsheet-oriented code and tool use; OfficeQA and DocVQA test local-document and multimodal-document reasoning; SealQA stresses noisy retrieval; LiveMathematicianBench evaluates mathematical multiple-choice reasoning; and ALFWorld tests sequential decision making. Dataset-backed benchmarks use deterministic train/selection/test splits, with a default 2:1:7 split when no benchmark-specific split is stated. The selection split is used only for model selection over candidate skills; all headline scores are computed on held-out test data.

Baselines. The no-skill baseline evaluates the frozen student without an optimized skill document. Human-skill and LLM-skill baselines use manually written and one-shot generated skills under

the same evaluation protocol. Trace2Skill mines skill artifacts from training trajectories and evaluates the frozen student without SKILLOPT’s iterative validation gate. TextGrad and GEPA are reflective prompt-optimization baselines for direct-chat settings. EvoSkill is included for the harness-backed comparison where a matched completed run is available. Entries not measured under the final aligned protocol are marked as –rather than mixed with incompatible runs.

Optimization protocol. Unless otherwise stated, SKILLOPT runs for four epochs with rollout batch size 40, reflection minibatch size 8, textual learning rate 4, cosine learning-rate decay with minimum rate 2, held-out validation gating, slow update enabled with 20 sampled examples, and optimizer-side meta skill enabled. The optimizer analyzes successes and failures separately, proposes patch-style skill edits, merges duplicate or contradictory proposals, ranks edits under the current learning-rate cap, and applies the selected edits to form a candidate skill. The candidate is evaluated on the selection split and is accepted only if it improves the current selection score; the best accepted skill is exported as `best_skill.md`. The student model, backend, harness, and benchmark evaluator remain fixed during optimization.

Ablation protocol. One-factor ablations vary a single scalar or component while holding the remaining optimizer configuration fixed. The train-size ablation fixes the train/selection/test split to 2:1:7 and varies how much of the training partition is exposed to the optimizer. The 100% row uses the full training partition under the same split, so it is directly comparable to the smaller-subset rows. Component ablations remove or alter one mechanism at a time, including the edit budget, rejected-edit buffer, and epoch-wise slow/meta update.

C.1 Optimization Procedure

Algorithm 1 expands the procedure used in the experiments. The central state variables are the current skill s_{cur} , the best validation-gated skill s_{best} , a selection-score cache $\mathcal{C}$, a step buffer $\mathcal{B}$ containing rejected edits and observed failure patterns, and an optimizer-side meta skill m_{meta} used only to guide future edit generation.

C.2 Optimizer Prompt Contracts

The following blocks reproduce the operational prompt contracts used by the optimizer model, with terminology normalized to the paper’s optimizer/training-model framing. The prompts require JSON outputs so that edits can be parsed, filtered, applied, and validated without manual intervention.

C.2.1 Failure analysis: `analyst_error.md`

You are an expert failure-analysis agent for AI agent tasks.

You will be given MULTIPLE failed agent trajectories from a single minibatch and the current skill document.

Your job is to identify the most important COMMON failure patterns across the batch and propose a concise set of skill edits.

Analysis Process

1. Read ALL trajectories in the minibatch.
2. Identify the most prevalent, systematic failure patterns across them.
3. For each pattern, classify its failure type.
4. Propose skill edits that address the COMMON patterns, not individual edge cases.
5. Edits must be generalizable; do not hardcode task-specific values.
6. Only patch gaps in the skill; do not duplicate existing content.

You will be told the maximum number of edits (the budget L). Produce AT MOST L edits,

Algorithm 1 SKILLOPT skill optimization

Require: Frozen training model M , optimizer model O , harness h , splits $D_{\text{train}}, D_{\text{sel}}, D_{\text{test}}$, initial skill s_0 , epochs E , edit-budget schedule L_t , rollout batch size B , accumulation factor A , reflection minibatch size B_m

Ensure: Best validation-gated skill s_{best} and held-out test score

```
1:  $s_{\text{cur}} \leftarrow s_0, s_{\text{best}} \leftarrow s_0, \mathcal{C} \leftarrow \emptyset, \mathcal{B} \leftarrow [], m_{\text{meta}} \leftarrow \emptyset$ 
2:  $\text{score}_{\text{cur}} \leftarrow \text{EVALUATE}(M, h, s_0, D_{\text{sel}}); \text{score}_{\text{best}} \leftarrow \text{score}_{\text{cur}}$ 
3:  $\mathcal{C}[\text{HASH}(s_0)] \leftarrow \text{score}_{\text{cur}}$ 
4: for  $e = 1$  to  $E$  do
5:   Shuffle  $D_{\text{train}}$  into rollout batches; reset  $\mathcal{B} \leftarrow []$ 
6:   for each optimization step in epoch  $e$  do
7:     Collect  $A$  rollout batches by executing  $h(M, x, s_{\text{cur}})$  for sampled tasks  $x$ 
8:     Split rollout evidence into failures and successes, then into minibatches of size  $B_m$ 
9:     Ask  $O$  to analyze failure minibatches and produce failure patch proposals
10:    Ask  $O$  to analyze success minibatches and produce success patch proposals
11:    Ask  $O$  to merge failure proposals, merge success proposals, and perform a final failure-
prioritized merge
12:    Ask  $O$  to rank merged edits and keep at most  $L_t$  edits
13:    Apply the selected edits to obtain a candidate skill  $\tilde{s}$ 
14:    if  $\text{HASH}(\tilde{s}) \in \mathcal{C}$  then
15:       $\text{score}_{\text{cand}} \leftarrow \mathcal{C}[\text{HASH}(\tilde{s})]$ 
16:    else
17:       $\text{score}_{\text{cand}} \leftarrow \text{EVALUATE}(M, h, \tilde{s}, D_{\text{sel}})$ 
18:       $\mathcal{C}[\text{HASH}(\tilde{s})] \leftarrow \text{score}_{\text{cand}}$ 
19:    end if
20:    if  $\text{score}_{\text{cand}} > \text{score}_{\text{cur}}$  then
21:       $s_{\text{cur}} \leftarrow \tilde{s}; \text{score}_{\text{cur}} \leftarrow \text{score}_{\text{cand}}$ 
22:      if  $\text{score}_{\text{cand}} > \text{score}_{\text{best}}$  then
23:         $s_{\text{best}} \leftarrow \tilde{s}; \text{score}_{\text{best}} \leftarrow \text{score}_{\text{cand}}$ 
24:      end if
25:    else
26:      Add rejected edits and observed failure patterns to  $\mathcal{B}$ 
27:    end if
28:  end for
29:  if  $e \geq 2$  and slow update is enabled then
30:    Compare the same sampled tasks under the previous and current epoch-end skills
31:    Ask  $O$  for protected longitudinal guidance; validate the injected guidance through  $D_{\text{sel}}$ 
32:  end if
33:  if  $e \geq 2$  and optimizer memory is enabled then
34:    Ask  $O$  to update  $m_{\text{meta}}$  for future edit generation and selection
35:  end if
36: end for
37:  $\text{score}_{\text{test}} \leftarrow \text{EVALUATE}(M, h, s_{\text{best}}, D_{\text{test}})$ 
38: return  $s_{\text{best}}, \text{score}_{\text{test}}$ 
```

focusing on the highest-impact patterns. You may produce fewer if warranted.

Respond ONLY with a valid JSON object (no markdown fences, no extra text):

```
{
  "batch_size": <number of trajectories analysed>,
  "failure_summary": [
    {"failure_type": "<type>", "count": <int>, "description": "<one-line>"}
  ],
}
```

```

"patch": {
  "reasoning": "<why these edits address the batch's common failures>",
  "edits": [
    {"op": "append",          "content": "<markdown to add at end of skill>"},
    {"op": "insert_after", "target": "<exact heading/text to insert after>",
     "content": "<markdown>"},
    {"op": "replace",        "target": "<exact text to replace>",
     "content": "<replacement>"},
    {"op": "delete",         "target": "<exact text to remove>"}
  ]
}

```

Only include edits that are needed. "edits" can be an empty list if no patch is warranted.

IMPORTANT: The skill document may contain a section between
 <!-- SLOW_UPDATE_START --> and <!-- SLOW_UPDATE_END --> markers.
 This is a PROTECTED section managed by a separate slow-update process.
 Do NOT propose any edits that target, modify, or delete content within these markers.

C.2.2 Success analysis: analyst_success.md

You are an expert success-pattern analyst for AI agents.

You will be given MULTIPLE successful agent trajectories from a single minibatch and the current skill document. Your job is to identify generalizable behavior patterns that are COMMON across the batch and worth encoding in the skill.

Rules

- Only propose patches for patterns NOT already covered in the skill.
- Focus on patterns that appear across MULTIPLE trajectories in the batch.
- Be concise. Patterns must generalize beyond specific tasks.
- Prefer reinforcing existing sections over adding new top-level sections.

You will be told the maximum number of edits (the budget L). Produce AT MOST L edits, focusing on the most broadly applicable patterns. You may produce fewer if warranted.

Respond ONLY with a valid JSON object:

```

{
  "batch_size": <number of trajectories analysed>,
  "success_patterns": ["<pattern 1>", "<pattern 2>"],
  "patch": {
    "reasoning": "<why these patterns are worth encoding>",
    "edits": [
      {"op": "append",          "content": "<markdown>"},
      {"op": "insert_after", "target": "<heading/text>", "content": "<markdown>"},
      {"op": "replace",        "target": "<old text>",      "content": "<new text>"},
      {"op": "delete",         "target": "<exact text to remove>"}
    ]
  }
}

```

"edits" may be empty if the skill already covers all observed patterns.

IMPORTANT: The skill document may contain a section between
 <!-- SLOW_UPDATE_START --> and <!-- SLOW_UPDATE_END --> markers.
 This is a PROTECTED section managed by a separate slow-update process.
 Do NOT propose any edits that target, modify, or delete content within these markers.

C.2.3 Failure merge: merge_failure.md

You are a skill-edit coordinator. You receive multiple independently-proposed patches from FAILURE analysis of agent trajectories. Merge them into ONE coherent,

non-redundant patch.

Merge guidelines:

1. Deduplicate: keep the best-worded version of similar edits.
2. Resolve conflicts: if patches contradict on the same point, choose the one with stronger justification or synthesize both.
3. Preserve unique insights: include all non-redundant corrective edits.
4. Prevalent-pattern bias: edits appearing consistently across multiple patches address systematic failures; preserve them with HIGH priority. Edits from only one patch may be discarded if task-specific.
5. Independence: no two edits in the merged patch may target the same text region.
6. Support count: for each merged edit, estimate how many source patches support it.
7. PROTECTED SECTION: The skill may contain a section between `<!-- SLOW_UPDATE_START -->` and `<!-- SLOW_UPDATE_END -->` markers. Do NOT merge or produce any edits that target content within these markers.

Respond ONLY with a valid JSON object:

```
{
  "reasoning": "<summary of key consolidation decisions>",
  "edits": [
    {
      "op": "append|insert_after|replace|delete",
      "target": "<if insert_after or replace or delete>",
      "content": "<markdown>",
      "support_count": <integer>,
      "source_type": "failure"
    }
  ]
}
```

C.2.4 Success merge: merge_success.md

You are a skill-edit coordinator. You receive multiple independently-proposed patches from SUCCESS analysis of agent trajectories. Merge them into ONE coherent patch that reinforces effective patterns.

Merge guidelines:

1. Deduplicate: keep only the most generalizable version of similar patterns.
2. Be conservative: success-driven patches reinforce existing behavior. Only include edits for patterns NOT already in the skill.
3. Prevalent-pattern bias: patterns seen across many successful trajectories are most worth encoding.
4. Support count: estimate how many source patches support each merged edit.
5. PROTECTED SECTION: The skill may contain a section between `<!-- SLOW_UPDATE_START -->` and `<!-- SLOW_UPDATE_END -->` markers. Do NOT merge or produce any edits that target content within these markers.

Respond ONLY with a valid JSON object:

```
{
  "reasoning": "<summary>",
  "edits": [
    {
      "op": "append|insert_after|replace|delete",
      "target": "<if needed>",
      "content": "<markdown>",
      "support_count": <integer>,
      "source_type": "success"
    }
  ]
}
```


C.2.5 Final merge: merge_final.md

You are a skill-edit coordinator performing the FINAL merge. You receive two pre-merged patch groups:

1. Failure-driven patches (corrective, high priority)
2. Success-driven patches (reinforcement, lower priority)

Merge guidelines:

1. FAILURE PATCHES TAKE PRIORITY: the primary goal of skill reflection is to fix failures. Failure-driven edits should be preserved unless they directly conflict with a well-supported success pattern.
2. Deduplicate: if a failure edit and success edit cover the same point, keep the failure version.
3. Preserve success insights: include success edits that cover patterns NOT addressed by failure edits.
4. Higher-level merges represent broader consensus: edits that survived previous merge rounds should be given priority.
5. Carry forward support_count and source_type for each edit.
6. PROTECTED SECTION: The skill may contain a section between `<!-- SLOW_UPDATE_START -->` and `<!-- SLOW_UPDATE_END -->` markers. Do NOT merge or produce any edits that target content within these markers.

Respond ONLY with a valid JSON object:

```
{
  "reasoning": "<summary of priority decisions>",
  "edits": [
    {
      "op": "append|insert_after|replace|delete",
      "target": "<if needed>",
      "content": "<markdown>",
      "support_count": <integer>,
      "source_type": "failure|success"
    }
  ]
}
```

C.2.6 Ranking and selection: ranking.md

You are an expert edit-ranking optimizer for a skill optimization system. You receive a skill document and a pool of proposed edits. Your job is to RANK the edits by importance and select the top ones.

Ranking criteria (in order of priority):

1. Systematic impact: edits that address widespread, recurring failure patterns across many tasks should rank highest. A rule that fixes 50% of failures beats one that fixes a single edge case.
2. Complementarity: edits that fill gaps in the current skill, not duplicate existing content, rank higher.
3. Generality: edits phrased as general principles rank higher than those tied to specific question types or entities.
4. Actionability: edits with clear, concrete guidance rank higher than vague advice.

You will be told how many edits to select (the budget).

Respond ONLY with a valid JSON object:

```
{
  "reasoning": "<brief justification for your ranking decisions>",
  "selected_indices": [<0-based indices of the top edits, in priority order>]
}
```

C.2.7 Slow update: slow_update.md

You are a strategic skill advisor for an AI agent optimization system.

Your role is different from the per-step analyst. The per-step analyst sees individual trajectories and proposes local patches. YOU see how the skill has evolved across an entire epoch by comparing the SAME tasks under two consecutive skill versions. This longitudinal view lets you identify systemic drift, regressions, and persistent blind spots that step-level edits cannot catch.

What You Receive

1. Previous epoch's skill and current epoch's skill, to see what changed.
2. Longitudinal comparison: the same 20 training tasks rolled out under both skills, categorized into regressions, persistent failures, improvements, and stable successes.
3. Previous slow update guidance, if any: the guidance written at the end of the last epoch.

Your Process

1. Reflect on the previous guidance, if provided:
 - Which parts of the previous guidance were effective?
 - Which parts failed or backfired?
 - Were there blind spots the previous guidance missed entirely?
2. Write updated guidance that:
 - Retains and strengthens parts of the previous guidance that proved effective.
 - Revises or removes parts that were ineffective or counterproductive.
 - Adds new instructions to address newly observed regressions and persistent failures.

Output Requirements

Write a strategic guidance block that will **OVERWRITE** the previous guidance in the protected section of the skill document. This section is **READ-ONLY** to all subsequent step-level optimization; only this epoch-boundary process can overwrite it at the next epoch boundary.

Your guidance must:

- Be written as direct, actionable instructions to the training model.
- Prioritize: (1) preventing regressions, (2) fixing persistent failures, (3) reinforcing successful patterns.
- NOT duplicate content already in the main skill body; complement it.
- Address the training model directly, for example: "When you encounter X, always do Y."

Respond **ONLY** with a valid JSON object:

```
{
  "reasoning": "<reflection on previous guidance AND analysis of longitudinal comparison>",
  "slow_update_content": "<the exact guidance text to insert into the protected section>"
}
```

C.2.8 Optimizer memory: meta_skill.md

You are an optimizer coach for an AI agent skill optimization system.

Your job is not to solve tasks directly and not to write training-model-facing skill rules. Your job is to write a compact optimizer-side meta skill that helps future optimizer calls produce better skill edits in this environment.

What You Receive

1. The previous epoch's last-step skill.
2. The current epoch's last-step skill.

3. A longitudinal comparison on the SAME sampled tasks under those two skills.
4. The previous optimizer memory, if one existed.

Your Goal

Write a concise optimizer memory that improves future optimizer behavior in stages such as failure analysis, success analysis, patch merging, and edit ranking.

This optimizer memory should capture things like:

- Which kinds of edits tend to help in this environment.
- Which kinds of edits tend to be too vague, redundant, brittle, or harmful.
- What level of abstraction works best for rules here.
- What failure-repair patterns should be prioritized.
- What regression risks future optimizer calls should guard against.

Important Constraints

- Address the FUTURE OPTIMIZER directly, not the training model.
- Focus on how to write better edits and organize better skill updates.
- Use evidence from the adjacent-epoch comparison, not generic advice.
- Keep it compact and high-signal. Prefer a few durable principles.
- Revise or remove parts of the previous optimizer memory if they did not help.
- Do not output training-model-facing task instructions.

Respond ONLY with a valid JSON object:

```
{
  "reasoning": "<brief reflection on what editing directions helped or hurt>",
  "meta_skill_content": "<compact optimizer guidance for future edits>"
}
```

C.3 Patch Representation and Safeguards

Patch-mode optimization restricts each update to four atomic operations: `append`, `insert_after`, `replace`, and `delete`. Each merged edit also records a support count and a source type, allowing ranking to prefer edits that survive independent analyses and hierarchical merges. The edit budget L_t acts as a textual learning rate: it limits how many proposed edits can be applied at a step, preserving continuity between adjacent skills.

The protected slow-update section, delimited by `SLOW_UPDATE_START` and `SLOW_UPDATE_END`, is off limits to all step-level prompts. Only the epoch-boundary slow-update process may rewrite that section, and the rewritten skill still passes through the same held-out selection gate before it can become the current skill. Rejected candidates are not discarded entirely: their failure patterns and rejected edits are stored in the step buffer so that later optimizer calls can avoid repeating harmful changes.

C.4 Design Principles

The implementation follows five design principles. First, the task-execution model is fixed; only the text skill changes. Second, every candidate skill is evaluated on a selection split before acceptance, which prevents unvalidated reflection from accumulating. Third, minibatch analyses are merged hierarchically so that the final edits represent recurring evidence rather than single examples. Fourth, the edit budget serves as a learning-rate analogue, allowing larger early changes and smaller late refinements. Fifth, the deployed skill remains lightweight and inspectable, while the optimizer-side meta skill stays separate from the skill shown to the task-execution model.